2795 一维数组 · 讲者备注

> 对照 2795-Arrays.pptx 逐页整理，内容为原视频（xmoj 竞赛 10058 问题 A「知识讲解-一维数组」）里有、PPT 上没有、讲课时可以补充的部分。括号里的时间戳是原视频中的位置，方便回看。

Slide 1 · 标题页（One-Dimensional Arrays）

【开场引入：为什么需要数组】（0:00–2:30）

我们之前学循环的时候做过一类题：读入 n 个数，算最大值、平均数。那时候我们是读一个、处理一个，下一次循环读进来的数会把上一个覆盖掉。循环结束，前面的数就全丢了，最多留下最后一个。

那如果循环结束以后，这些数我还要接着用呢？比如要把它们倒过来输出。

有同学会说：有几个数我就定义几个变量。想一想：题目说最多 100 个，你定义 100 个变量？说 1000 个呢？而且程序也没法写——你在循环里要写"如果是第 1 次就存到第 1 个变量，第 2 次存到第 2 个变量……"，这根本写不出来。

所以我们需要一种新的办法，一次性定义一大批变量，这就是今天要学的数组。

Slide 2 · 例题：反向输出（先抛问题）

先不急着讲代码，就用这道题当引子："要倒过来输出，就必须把所有数都记住"——带着这个需求进入下一页的语法。

Slide 3 · 语法：定义与引用

【"房间"比喻，建议边说边在白板画图】（2:40–5:20）

把内存想象成一排房间。以前写 `int a`，是让计算机开一个 4 字节的房间，门牌叫 a。现在写 `int a[100]`，是让计算机连续开 100 个这样的房间。

注意：这 100 个房间没有各自的名字，全都叫 a，靠编号区分。编号从 0 开始，最后一个是 99，不是 100。

【同一个方括号，两种意思】（6:27）

定义的时候 `int a[100]`，方括号里的 100 叫"元素个数"；使用的时候 `a[15] = 15`，方括号里的 15 叫"下标"，表示第 16 个房间。定义和引用是两件不同的事，千万别混。

【越界不报错】（7:30–10:05）

开了 100 个房间，能不能写 `a[100]`？C++ 不会报错，代码是合法的。但你其实是往"隔壁房间"放了东西，而隔壁房间里装的是什么你根本不知道，可能是程序运行要用的重要数据，像个弹药库。这叫"数组越界"。

我们之前强调过两个最容易犯的错：一是 `=` 和 `==` 搞混，二是变量没初始化就用。数组越界是第三个，而且它不报错，程序只会莫名其妙地出问题，查半天都查不出来。后面我们会现场演示。

【为什么不能写 `int a[n]`】（10:20–11:35）

其实这么写编译也能过，但我们要求大家不要写。原因是：这段代码在函数里面，函数能用的内存很有限；而 n 是从黑窗口输入的，你控制不了。万一输进来一个很大的 n，程序直接崩掉。所以数组大小一定要写固定的数，按题目给的最大范围来开。

【能开多大？让学生自己算】（12:00）

题目说 n 最大 10^7，能开 `int a[10000000]` 吗？大家算一下：一个 int 4 字节，10^7 个就是 4×10^7 字节，大约 38 MB。没问题，可以开。现阶段题目怎么约定就怎么开，空间优化是以后学算法的事。

【const 是什么意思】（13:05）

PPT 上的 `const int MAXN=110;`，const 是"常量"的意思：定义的时候给了值，以后一辈子都不允许改。所以它可以用来当数组大小，效果和直接写数字一样。

Slide 4 · 例题：反向输出（讲解代码）

【在白板上模拟一遍】（15:30–18:15）

假设输入 5，再输入 1 2 3 4 5。i=0 时读到 1，放进 a[0]；i=1 时读到 2，放进 a[1]……5 个数依次放进前 5 个房间。后面 95 个房间我们没碰过，是空的，也不能去读。

【为什么倒序循环从 n-1 开始】（18:20）

刚才正着读的循环，最后一次条件成立时 i 是几？是 n-1。i 再加 1 变成 n，条件就不成立了。所以倒着输出从 n-1 开始，到 0 也要输出，条件写 `i>=0`，每次 `--i`。

【常见错误：`cin >> a;`】（21:25）

有同学会想：a 是一堆变量，输入也是一堆数，我直接 `cin >> a;` 一次读完。想法很朴素，但做不到。每读一个数放到哪个房间，必须明确告诉计算机，所以一定要用循环。

【最后多一个空格要不要紧】（22:30）

每次输出数字加空格，最后会多一个空格。在 NOIP 这类评测里，最后一行末尾多一个空格或多一个换行是可以容忍的。注意：只有最后一行有这个待遇，不是每一行后面都可以多空格。

【为什么数组要多开 10 个】（23:45–24:55）

题目说 n 最大 100，我们开 110。为什么？比如你开了 `a[100]`，循环写成 `i=1; i<=n`，当 n 正好是 100 时，最后一次就访问到 `a[100]`，越界了。多开 10 个，不管下标从 0 开始还是从 1 开始都安全。

Slide 5 · 例题：斐波那契数列

【先编号再写公式】（25:05）

在数列上方标上序号：第 1 个、第 2 个……从第 3 个开始，每个数等于前两个数之和。数组在内存里是连续的，所以 a[i] 的前两个就是 a[i-1] 和 a[i-2]：`a[i] = a[i-1] + a[i-2]`。用循环来做要倒来倒去好几个变量，有了数组就简单多了。

【a[0] 不用，这是一个常用技巧】（27:00）

数组下标规定从 0 开始，但我们完全可以把 a[0] 扔掉，让下标从 1 开始，和题目里"第几个"对应起来。代价只是数组多开一点。

【补充一句数据类型】

40 以内用 int 够了，第 40 项是 102334155。但大概从第 47 项开始就超出 int 了，需要用 long long（原视频代码用的就是 `long long a[51]`）。

Slide 6 · 语法：初始化

【用循环初始化】（28:45）

数组开了这么多房间，初始化绕不过去。最直接的办法是写个循环：`a[i]=0` 全部清零，`a[i]=i` 放成 0,1,2…，`a[i]=i+1` 放成 1,2,3…，想怎么初始化都行。下面讲几种标准写法。

【`{0}` 这个写法】（30:50）

讲完"不够的补 0"之后，问学生：如果我想全部清零，是不是可以写 `int a[100] = {0};`？——答对了，可以。不过实际上大家不太这样写，因为还有更方便的写法。

【花括号只能在定义时用】（31:15）

先写 `int a[100];`，下一行再写 `a = {0};`，这是不行的。花括号初始化必须和定义写在同一行。

【什么时候用 memset】（31:45）

比如在一个循环里，每一轮都要把数组重新清零（多组测试数据就常这样）。难道每次都写一个循环去清？不用，一句 `memset(a, 0, sizeof(a));` 就行。

【memset 为什么不能设成 1】（33:40，画图）

画出来：一个 int 是 4 个字节，memset 是往每一个字节里填你给的数。填 0，4 个字节全是 0，整个 int 就是 0。填 1，是 4 个字节各自变成 1，拼起来就不是十进制的 1 了（实际是 16843009，学生问起再说）。所以 memset 只用 0 或 -1；想全部设成 1，自己写循环。

【局部变量 vs 全局变量】（34:47–36:56）

平时我们把变量写在 main 函数里，这叫局部变量，初始值是随机的。写在 main 函数外面的叫全局变量，系统会自动把它全部初始化为 0，连 memset 都不用写。

有同学会说：老师你怎么不早说？我是故意放到现在才讲的——希望大家先牢牢记住"变量不初始化就不能用"。全局变量自动清零，不代表不用初始化，而是系统用你看不见的方式帮你做掉了。

Slide 7 · 例题：一年中的第几天

【建议分两步讲，先 a[12] 再 a[13]】（38:40–42:30）

先引导：每个月天数没规律，但规律在于"把这个月之前的整月天数全加上，再加当月的日"。没规律的天数，就用数组记下来。

第一版：开 `a[12]`，1 月存在 a[0]，2 月存在 a[1]……循环 i 从 1 到 m-1 时要写 `ans += a[i-1]`，这个减 1 很容易搞错。

第二版（也就是 PPT 上的写法）：开 `a[13]`，a[0] 放个 0 不用，1 月就是 a[1]，月份就是下标，减 1 就去掉了，程序更直观。浪费或跳过几个元素来让代码更清楚，这是常用技巧。

【闰年】（40:23）

2016 年是闰年，2 月有 29 天，为什么数组里写 28？先按 28 算，最后再判断：如果是闰年并且月份大于 2，再加 1 天。当月是 2 月就不用加。

【⚠️ PPT 代码有个 bug，讲之前先改】

这一页两段代码里 `for (i=1; ...)` 的 i 都没有声明，编译会报错，要改成 `for (int i=1; ...)`。

Slide 8 · 例题：数组越界（建议换成视频版本现场运行）

【建议改代码】（42:35–44:10）

视频用的是：

```
int i;
int a[10];
for (i=1; i<=15; ++i){
    a[i] = 0;
    cout << i << " ";
}
```

讲解：数组只有 10 个元素，有效下标 0 到 9，从 a[10] 开始就越界了，一直写到 a[15]。现场运行，结果是死循环。

不用展开讲原因，让学生直观感受一下：你以为只是往隔壁房间放了点东西，结果那里可能就是个定时炸弹。（如果学生追问：很可能是越界写入把 i 改回了 0，但这和编译器有关，课前先自己跑一下确认。）

现在 PPT 上的版本输出的是 a[i]，可能只会看到一串 0，学生看不出有什么问题，所以建议换成这个版本。

Slide 9 · 例题：统计数字出现次数

【复习拆数位】（46:15）

不断用 `%10` 取个位，再 `/10` 把个位去掉，原来的十位就变成新的个位。一直除到 0 为止，8 位数就循环 8 次。

【点明"计算结果直接当下标"】（49:28）

`++a[n%10]`：拆出来的数位一定在 0 到 9 之间，正好拿来当下标，找到负责计数的那个房间，让它加 1。这种"用算出来的结果当下标"的写法很紧凑，也非常常见，就算自己不写，也会经常看到，要习惯。

【再强调一次初始化】（48:28）

这里 a 定义成全局变量，所以自动是 0。如果你把它写在 main 里面，就一定要写 `={0}` 或者 memset。

Slide 10–11 · 结束页 / One More Thing

视频里没有对应内容，按 PPT 讲即可。